CLASS 4

Cache: Making Applications Faster

A beginner-friendly introduction before we add cache to our Django CRM.

CACHE

TODAY

What will we learn?

Understand

What cache is and why applications need it.

Recognize

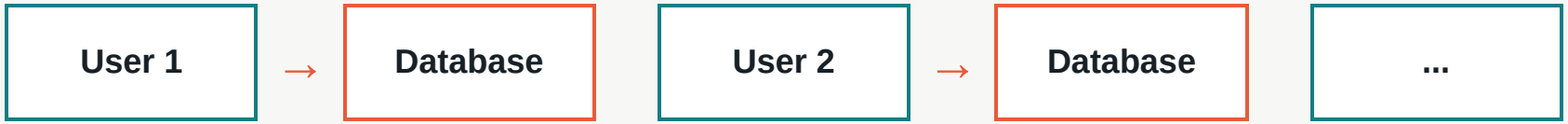
Key words, types, and the data cache can store.

- We will **not** write code today.
- Tomorrow, we will add a small cache to our CRM.

OUR CRM PROBLEM

What if every request asks the database?

Imagine 1,000 users opening the same customer list at the same time.



The database repeats the **same work** again and again.

Simple classroom assumption: one customer-list query needs 10 ms of database time. The database can safely complete about 100 such queries per second.

Users arrive together	Repeated database reads	Approx. time to finish them all
1 user	1 query	10 ms
1,000 users	1,000 queries	1,000 ÷ 100/sec = **10 seconds**
10,000 users	10,000 queries	10,000 ÷ 100/sec = **100 seconds**

Later requests wait in a queue. In a real system, heavy pressure can make the time even worse.

MEANING

What is cache?

Cache is a temporary, fast copy of data that we may need again soon.

Temporary

It can expire or be removed.

Fast

Usually quicker than asking the database.

A copy

The main data still belongs in the database.

EVERYDAY IDEA

Desk versus library

Without cache

Each time you need one book, walk to the library and find it.

With cache

Keep today’s most-used book on your desk for quick access.

The library is like the **database**. Your desk is like the **cache**.

WITHOUT CACHE

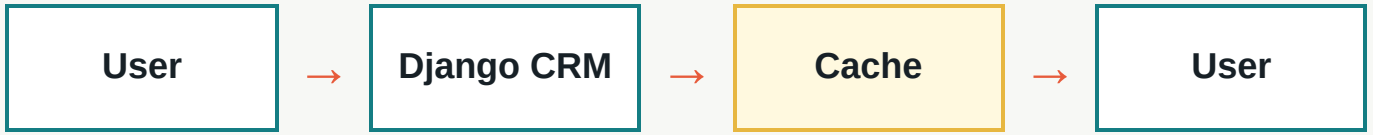
Normal request flow



Every request waits for the database query.

WITH CACHE

First ask the fast copy



Only when the cache does not have the data, Django asks PostgreSQL and saves a copy for later.

BASIC STRUCTURE

Cache usually uses a key and a value

```
Key:   "customer_list"
Value: [customer data]
```

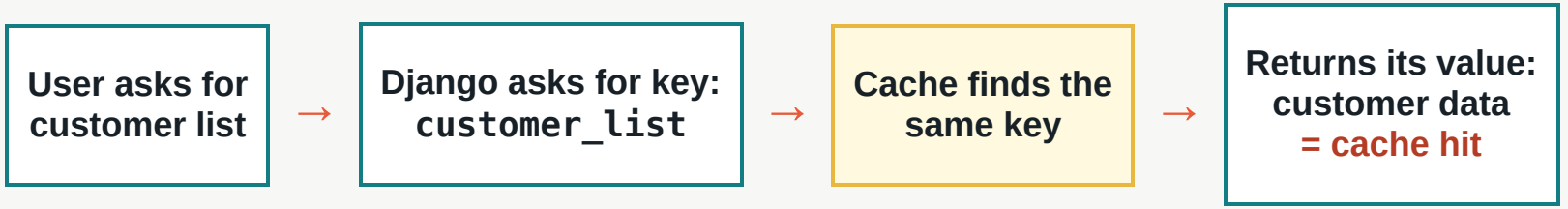
Key

A unique label used to find cached data.

Value

The saved data: text, number, image, JSON, or more.

Example: A user asks, “Show me the customer list.” Django uses the key `customer_list` to ask cache.



The **key** finds the data. The connected **value** is what cache returns.

TWO POSSIBLE RESULTS

Cache hit and cache miss

Cache hit ✓

Data is found in cache.
Return it quickly.

Cache miss ✗

Data is not in cache.
Ask the database.

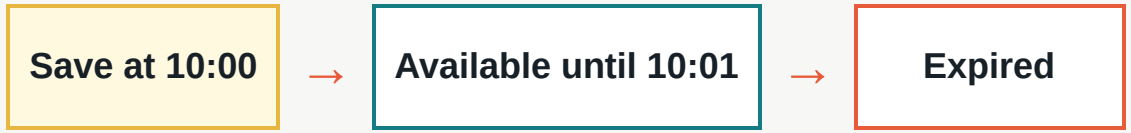
Cache miss → **Get from database** → **Save in cache**

IMPORTANT WORD

TTL: Time To Live

TTL tells cache how long it may keep a value.

```
customer_list  ->  keep for 60 seconds
```



After expiry, Django gets fresh data from the database again.

WHERE IT LIVES

How is cache stored?

Memory / RAM

Very fast. Common for application cache.

Disk

Can store more, but is usually slower than RAM.

Near the user

A browser or CDN may store files close to the user.

How much space? There is no one “typical” size. The developer sets a limit. For our classroom example, imagine a **100 MB** memory cache.

Limit: 100 MB → **Save data: 40 MB used** → **More data: 95 MB used** → **Full: may remove old values**

Used cache space **does** go up when we save values. It goes down when values expire (TTL), are deleted, or are removed to make space. The exact full-cache rule depends on the cache. The limit stays 100 MB until we change it.

TYPES: WHERE IT LIVES

Cache can exist in different places

Type	Location	Simple example
Browser cache	User's browser	Logo image already downloaded
CDN cache	Servers near users	Website image or video
Application cache	With/near Django	Customer list or dashboard total
Database cache	Database system	Frequently read query data

CDN = Content Delivery Network. It is a network of servers that delivers files, such as images and videos, from a server close to the user.

Real-world connection:

Netflix = CDN cache

When you press Play, Netflix can stream video from a suitable nearby/ISP server that already stores a copy.

TYPES: WHAT IT STORES

Cache can store many kinds of data

Words & numbers

Customer name, product price, dashboard count.

JSON / API response

A customer list returned by an API.

Images & files

Profile image, CSS, JavaScript, PDF, or video.

So “cache type” can mean **where it lives** or **what data it stores**.

Examples as cache entries (key → value):

`customer_42_name`
→ `Rahim`

`customer_42`
→ `{"id": 42, "name": "Rahim"}`

`customer_42_profile_image`
→ Rahim's profile image file

WHY USE IT?

Cache helps the whole system

Faster

Users get common data sooner.

Less database work

Fewer repeated queries.

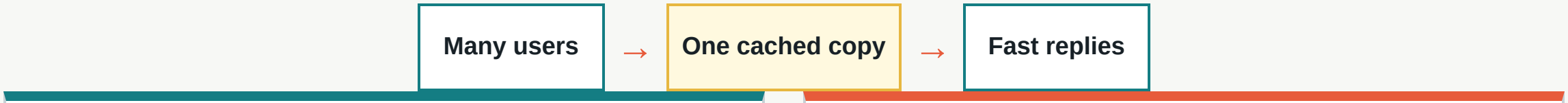
More users

The system can handle more repeated reads.

Cache is especially useful for data that is read often but changes less often.

SYSTEM DESIGN

Cache protects the database during heavy traffic



Without cache

Many repeated reads reach the database.

With cache

Many reads can use the same cached value.

REAL EXAMPLE

How Facebook used cache while scaling

Facebook began using **Memcached** in 2005 because database queries on every page load were making pages slower.

The idea

Keep frequently requested data in fast memory instead of repeatedly asking the database.

The result

Cache became an important part of handling huge numbers of requests as Facebook grew.

Simple illustration (not Facebook's exact code): Many people open the same popular profile or post.



Source: Meta Engineering, “Scaling memcache at Facebook,” 2013 — engineering.fb.com/2013/04/15/core-infra/scaling-memcache-at-facebook/

IMPORTANT RULE

Cache is not the main database

Database = official record

Keep the important, permanent customer and bill data here.

Cache = fast temporary copy

May expire, be cleared, or sometimes contain older data.

If cache disappears, the application should still work by reading the database.

BE CAREFUL

Cache has trade-offs

Old data

A cached value may not show a recent update.

Memory limit

We cannot save everything forever.

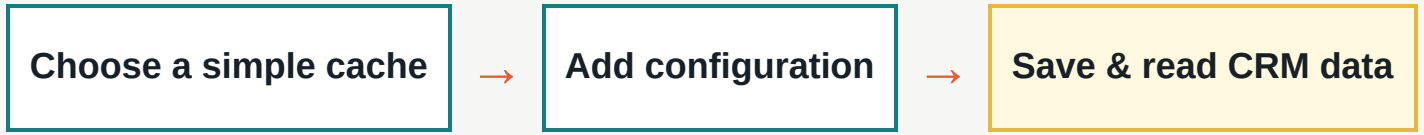
Extra thinking

We must decide when to refresh or remove it.

The right question is: **what is safe and useful to cache?**

TOMORROW

Cache in our Django CRM



- Which cache type will our CRM use?
- Why choose it now?
- What do we gain, and what do we give up?

QUICK REVIEW

Can you answer these?

1. What is the difference between a cache hit and a cache miss?
2. Why does cache use a key?
3. Why is a TTL useful?
4. Why is cache helpful, but not a replacement for the database?

Remember: cache keeps commonly needed data closer and faster.